PROJECT REPORT

AI HR Recruitment Assistant

An Explainable, Agentic-AI Powered Recruitment Support System

TNSDC - IBM Agentic AI Internship

PROJECT SUBMISSION

Name :M.Thanalakshmi

Reg no :960523205029
Clg name : Cape Institute of Technology

Clg code : 9605

1. Project Overview

1.1 Project Title

AI HR Recruitment Assistant — An Agentic AI-based system for explainable, human-reviewed candidate screening and interview preparation.

1.2 Problem Statement

Manual resume screening is slow, inconsistent, and prone to unconscious bias. HR teams receive large volumes of resumes for every job opening and must manually compare each candidate against the job description, extract relevant skills and experience, and decide who to shortlist. This process consumes significant time, lacks a transparent and repeatable scoring method, and offers little support for preparing candidate-specific interview questions. There is a need for a system that speeds up screening while keeping the process transparent, fair, and firmly under human control.

1.3 Brief Description of the Project

The AI HR Recruitment Assistant is a full-stack web application that assists HR personnel throughout the recruitment pipeline. HR users upload a job-description PDF and multiple candidate resume PDFs. The system extracts structured information from both, retrieves supporting context from a knowledge base using Retrieval-Augmented Generation (RAG), computes a transparent, weighted match score for every candidate, generates a shortlist/review/reject recommendation, and creates personalized interview questions targeting each candidate's skill gaps. The entire workflow is orchestrated as a multi-step agentic pipeline using LangChain and LangGraph, and every decision is explainable and advisory — the final hiring decision always rests with the HR user.

2. Objectives & Proposed Solution

2.1 Project Objectives

- **Automate resume-to-job matching:** reduce the manual effort of comparing candidate resumes against job requirements.
- **Ensure explainability:** produce scores and recommendations that HR can audit, understand, and trust.
- **Ensure fairness:** exclude protected/sensitive attributes (e.g. religion, gender, caste, disability, marital status) from all scoring logic.
- **Support human-in-the-loop decisions:** position AI output strictly as advisory, keeping HR as the final decision-maker.
- **Assist interview preparation:** auto-generate personalized interview questions that probe a candidate's missing or weak skill areas.
- **Enable knowledge-grounded responses:** use a RAG knowledge base of company, recruitment and interview guidelines to ground AI outputs.

2.2 How the Agentic AI Solution Works

The system models recruitment as a sequential agentic workflow built with LangGraph, where each stage is a node in a state graph and the output of one stage feeds the next:

- Job Analysis — parses the uploaded job-description PDF and extracts required skills, experience, and responsibilities.
- Resume Analysis — extracts candidate name, contact details, skills, education, experience and projects from the uploaded resume PDF.
- RAG Retrieval — queries a ChromaDB vector store of recruitment/interview guidelines to fetch context relevant to the required skills.
- Candidate Matching — computes a transparent, weighted score comparing candidate data against job requirements.
- Recommendation — classifies the candidate as SHORTLIST, REVIEW, or NOT RECOMMENDED based on the score.
- Interview Question Generation — uses a configurable LLM (via LangChain) to generate questions targeted at the candidate's missing skills, with a deterministic fallback when no LLM key is configured.
- Report Generation — compiles the score, explanation, matched/missing skills and RAG sources into a final candidate report.

LangChain tools wrap each capability (PDF parsing, knowledge-base search, scoring, question generation) so the agent can invoke them individually or as part of the LangGraph pipeline, keeping every step inspectable and independently testable.

2.3 Key Features

- Secure HR registration and login using JWT-based authentication.
- Job-description and multi-resume PDF upload with structured text extraction (PyMuPDF).
- Explainable, weighted candidate scoring with a clear breakdown by criterion.
- Automatic SHORTLIST / REVIEW / NOT RECOMMENDED recommendation.
- RAG-based knowledge retrieval from an ingestible knowledge base (company info, recruitment guidelines, interview guidelines, skill descriptions).
- Personalized interview-question generation based on each candidate's missing skills.
- Candidate report generation accessible via API.
- Fairness safeguard: protected/sensitive attributes are detected and excluded from scoring.
- Responsive React dashboard for HR to manage jobs and review ranked candidates.
- Automated backend tests covering extraction and scoring logic.

3. Implementation & Results

3.1 Technologies / Tools Used

Layer	Technology / Tool
Frontend	React 18, Vite, React Router, Axios
Backend	FastAPI, Python, Uvicorn, Pydantic
Database	PostgreSQL 18, SQLAlchemy ORM
Authentication	JWT (python-jose), Passlib/bcrypt password hashing
Document Processing	PyMuPDF (PDF text extraction)
Agentic AI Orchestration	LangChain (tools), LangGraph (stateful workflow)
LLM Integration	OpenAI API via langchain-openai (configurable, optional)
Knowledge Retrieval (RAG)	ChromaDB vector store
Testing	Pytest, httpx

3.2 Working Process

The end-to-end pipeline follows the sequence:

Job PDF → Extraction → Job Analysis → Resume Analysis → RAG Retrieval → Candidate Matching → Transparent Scoring → Recommendation → Interview Questions → Report

Step-by-step process:

- An HR user registers/logs in and creates a job posting by uploading a job-description PDF; the backend extracts required skills, experience and responsibilities.
- The HR user uploads one or more candidate resumes for that job; each PDF is parsed and structured into name, contact details, skills, education, experience and projects.
- The RAG service retrieves relevant guideline snippets from the ChromaDB knowledge base based on the job's required skills.
- The matching service compares candidate data to job requirements and computes a weighted overall score, listing matched and missing skills.
- Based on the score, the system assigns a recommendation label and generates an explanation string describing how the score was derived.
- The interview module generates candidate-specific questions targeting missing skills, using an LLM when configured, or a rule-based fallback otherwise.
- All results are exposed through REST APIs and rendered on the HR dashboard for review; the HR user makes the final call on each candidate.

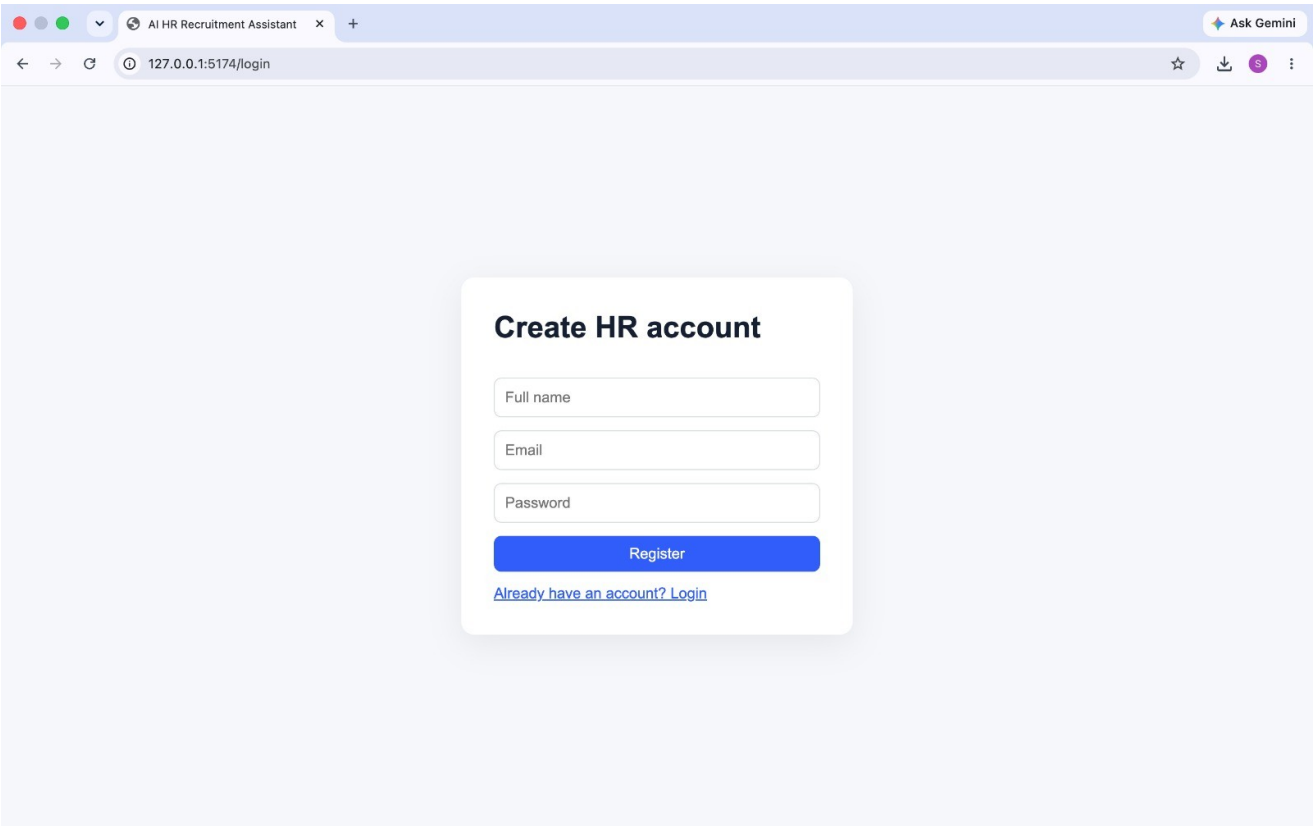
3.3 Screenshots / Output

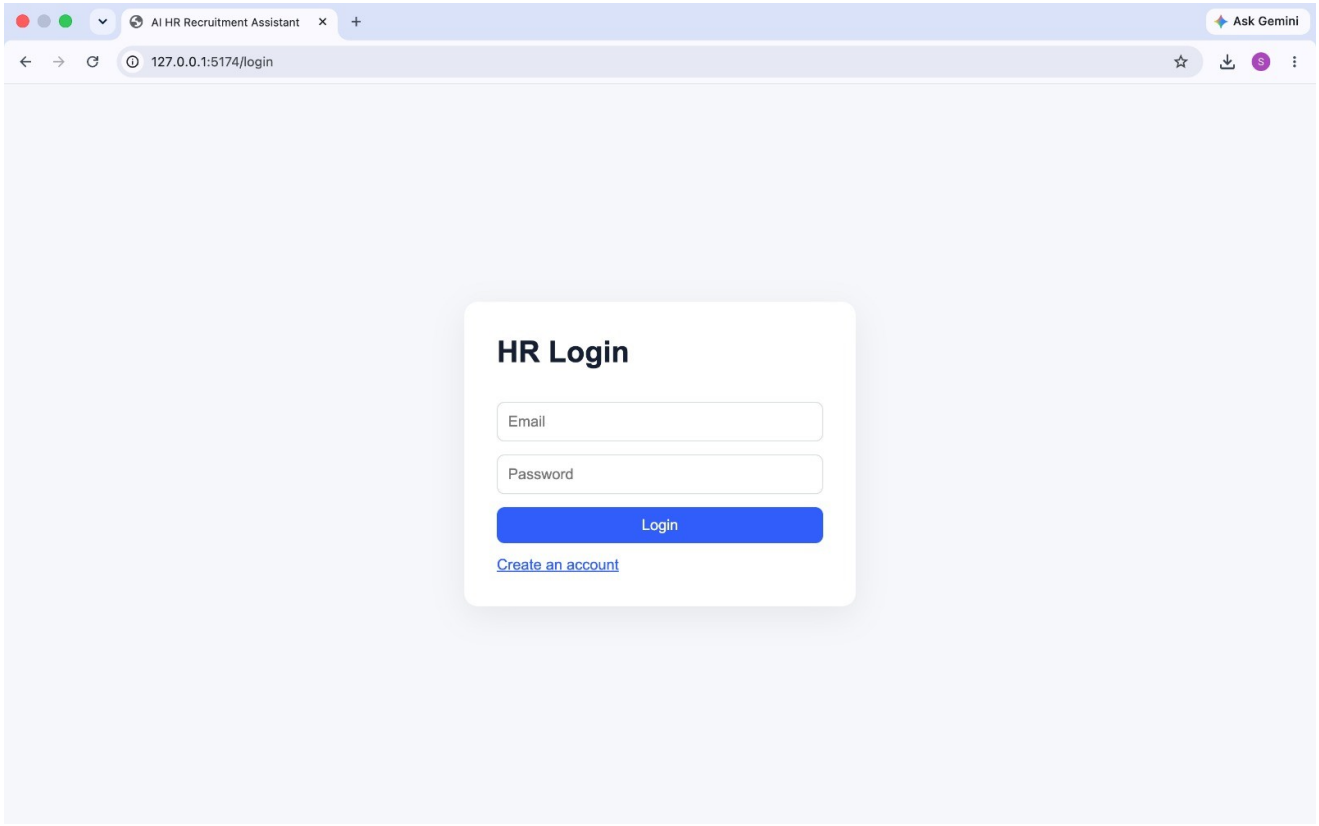
Representative outputs produced by the system include:

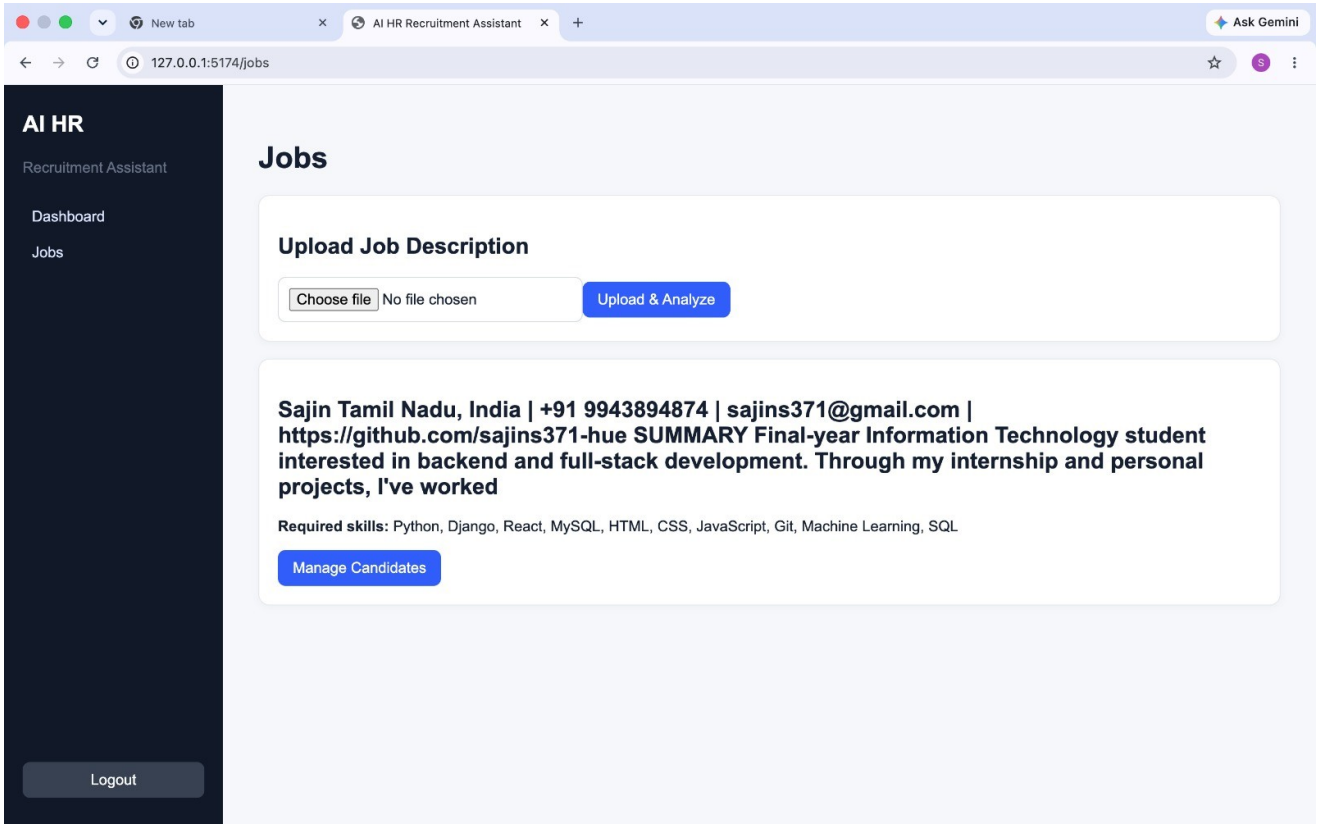
- HR dashboard listing jobs and candidate counts.
- Ranked candidate list per job, ordered by overall match score.

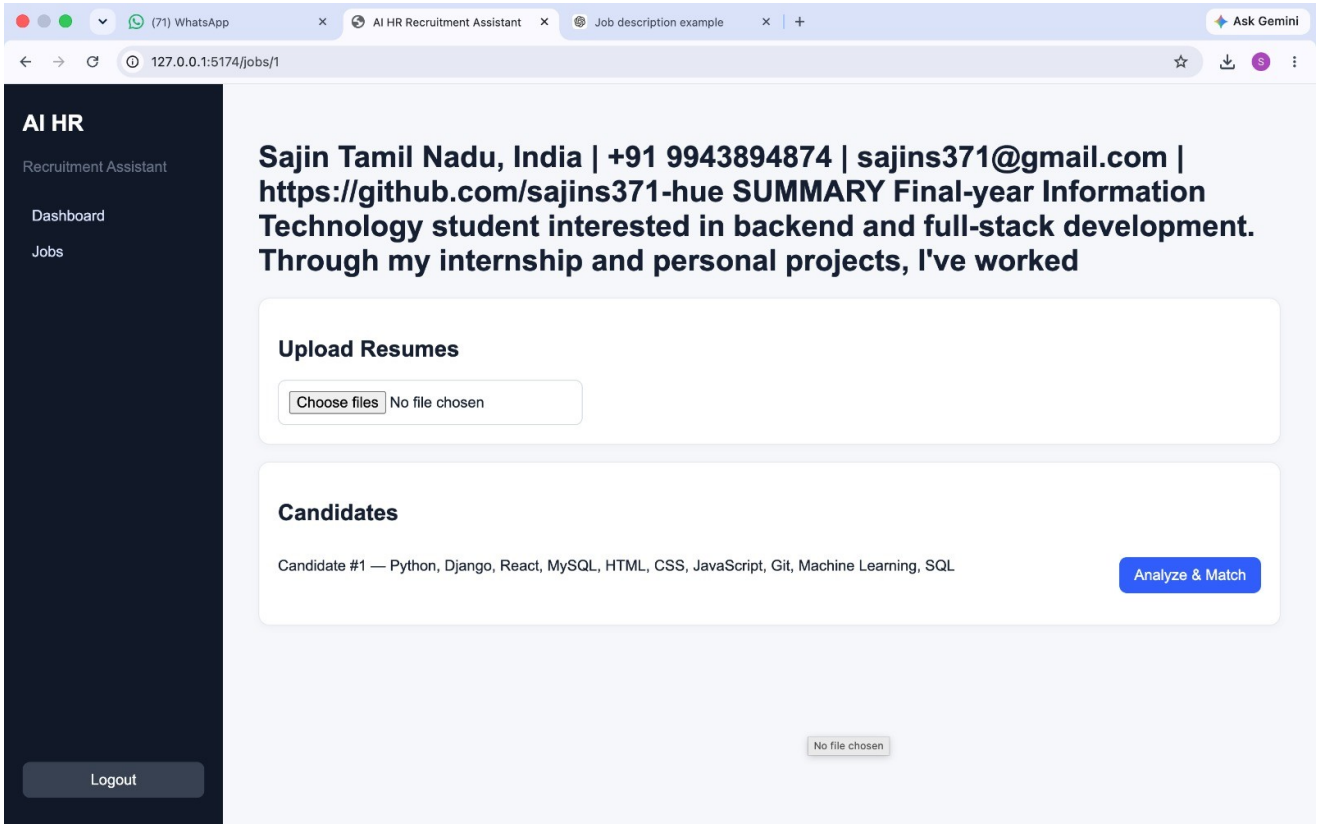
- Candidate detail view showing the score breakdown, matched/missing skills and recommendation label.
- Generated interview-question list targeted at a candidate's skill gaps.
- Candidate analysis report combining score, explanation and RAG-sourced context.

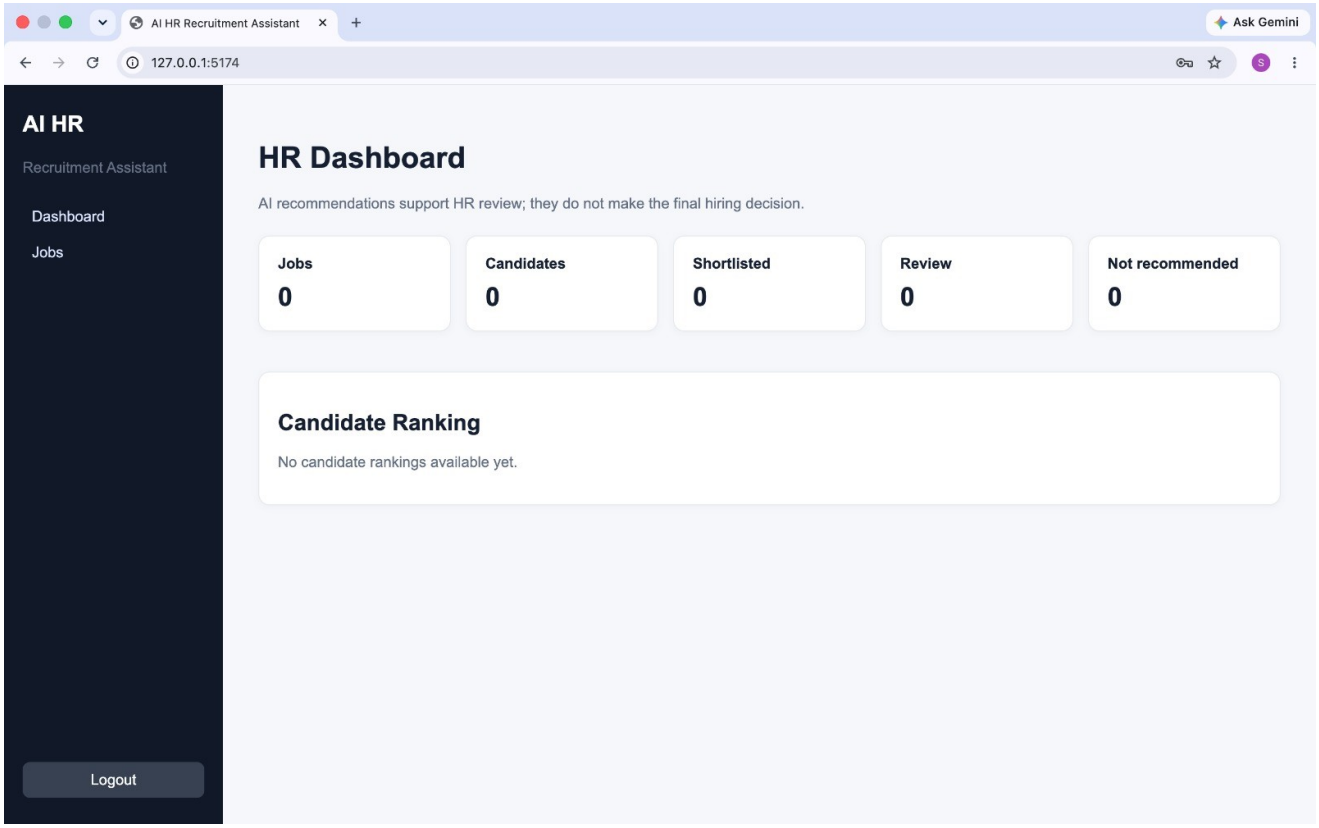
Output images:




AI HR Recruitment Assistant
Ask Gemini
127.0.0.1:5174/login
HR Login
Email
Password
Login
Create an account


New tab
AI HR Recruitment Assistant
Ask Gemini
127.0.0.1:5174/jobs
AI HR
Recruitment Assistant
Dashboard
Jobs
Logout
Jobs
Upload Job Description
Choose file
No file chosen
Upload & Analyze
Sajin Tamil Nadu, India | +91 9943894874 | sajins371@gmail.com | https://github.com/sajins371-hue SUMMARY Final-year Information Technology student interested in backend and full-stack development. Through my internship and personal projects, I've worked
Required skills: Python, Django, React, MySQL, HTML, CSS, JavaScript, Git, Machine Learning, SQL
Manage Candidates


(71) WhatsApp
AI HR Recruitment Assistant
Job description example
Ask Gemini
127.0.0.1:5174/jobs/1
AI HR
Recruitment Assistant
Dashboard
Jobs
Logout
Sajin Tamil Nadu, India | +91 9943894874 | sajins371@gmail.com | https://github.com/sajins371-hue SUMMARY Final-year Information Technology student interested in backend and full-stack development. Through my internship and personal projects, I've worked
Upload Resumes
Choose files
No file chosen
Candidates
Candidate #1 — Python, Django, React, MySQL, HTML, CSS, JavaScript, Git, Machine Learning, SQL
Analyze & Match
No file chosen


AI HR Recruitment Assistant
Ask Gemini
127.0.0.1:5174
AI HR
Recruitment Assistant
Dashboard
Jobs
Logout
HR Dashboard
AI recommendations support HR review; they do not make the final hiring decision.
Jobs
0
Candidates
0
Shortlisted
0
Review
0
Not recommended
0
Candidate Ranking
No candidate rankings available yet.

3.4 Results Achieved

The scoring model is fully transparent and reproducible. Each candidate's overall score is computed as a weighted sum across five criteria:

Criterion	Weight	Score Basis	Notes
Technical Skills	40%	Skill overlap	Required skills matched vs. job description
Experience	25%	Presence/absence	Years and role evidence extracted from resume
Education	15%	Presence/absence	Degree/qualification match against requirement
Projects	10%	Presence/absence	Relevant project evidence in resume
Certifications	10%	Presence/absence	Relevant certification evidence in resume

The overall score maps to a recommendation as follows:

Score Range	Recommendation	Meaning
80 - 100	SHORTLIST	Strong job-relevant match; proceed to interview
60 - 79.99	REVIEW	Partial match; HR review recommended
Below 60	NOT RECOMMENDED	Weak job-relevant match

Key working APIs implemented and verified through automated tests:

Endpoint	Purpose
POST /api/auth/register, /login	HR user registration and JWT-based login
GET/POST /api/jobs, /api/jobs/upload	Create jobs and upload job-description PDFs
POST /api/jobs/{id}/resumes	Upload one or more candidate resume PDFs
GET /api/jobs/{id}/candidates	List and rank candidates for a job
POST /api/candidates/{id}/analyze	Run extraction, matching and scoring for a candidate
POST/GET /api/interview/candidates/{id}/generate	Generate personalized interview questions
POST/GET /api/reports/candidates/{id}	Generate/retrieve the candidate analysis report
GET /api/dashboard	HR summary dashboard data
GET/POST /api/knowledge-base, /ingest	Manage and ingest RAG knowledge-base documents

Automated tests (pytest) validate the core resume/job extraction logic and the scoring function, and the fairness rule was verified to exclude protected attributes (religion, caste, gender, sex, disability, marital status, race, ethnicity, political affiliation) from every scoring path.

4. Conclusion & Future Scope

4.1 Project Conclusion

The AI HR Recruitment Assistant demonstrates how an agentic AI workflow — built with LangChain and LangGraph — can meaningfully support, rather than replace, human recruiters. By combining deterministic, explainable scoring with LLM-assisted interview-question generation and RAG-grounded knowledge retrieval, the system reduces manual screening effort while keeping every decision transparent and auditable. The explicit exclusion of protected attributes from scoring logic ensures the tool supports fair, job-relevant evaluation, and the human-in-the-loop design keeps the final hiring decision with the HR team at all times.

4.2 Challenges Faced

- Reliably extracting structured information (skills, experience, education) from unstructured and inconsistently formatted resume PDFs.
- Designing a scoring formula that is both simple to explain to non-technical HR users and fair across varied candidate profiles.
- Ensuring the system remains fully functional (PDF extraction, deterministic scoring, dashboard) even when no external LLM API key is configured.
- Explicitly identifying and excluding protected/sensitive attributes from every stage of the pipeline to guarantee fairness.
- Coordinating state across a multi-step LangGraph workflow while keeping each node independently testable.

4.3 Future Enhancements

- Add resume parsing support for additional formats (DOCX, scanned/OCR resumes).
- Introduce bias-audit dashboards to continuously monitor scoring fairness across candidate demographics.
- Support multi-language resumes and job descriptions.
- Add richer education/certification/project extraction using LLM-based parsing instead of purely regex-based extraction.
- Introduce role-based access control for multiple HR reviewers collaborating on the same job.
- Add analytics on hiring-funnel outcomes to refine the scoring weights over time.
- Deploy the system with containerization (Docker) and CI/CD for production readiness.

4.4 References

- FastAPI Documentation — https://fastapi.tiangolo.com

- LangChain Documentation — https://python.langchain.com
- LangGraph Documentation — https://langchain-ai.github.io/langgraph
- ChromaDB Documentation — https://docs.trychroma.com
- PyMuPDF (fitz) Documentation — https://pymupdf.readthedocs.io
- React Documentation — https://react.dev
- PostgreSQL Documentation — https://www.postgresql.org/docs
- OpenAI API Documentation — https://platform.openai.com/docs

Github repository link:

https://github.com/mthamm2812-lang/AI-HR-RECRUITMENT-ASSISTANT.git